

Distribuirani sistemi

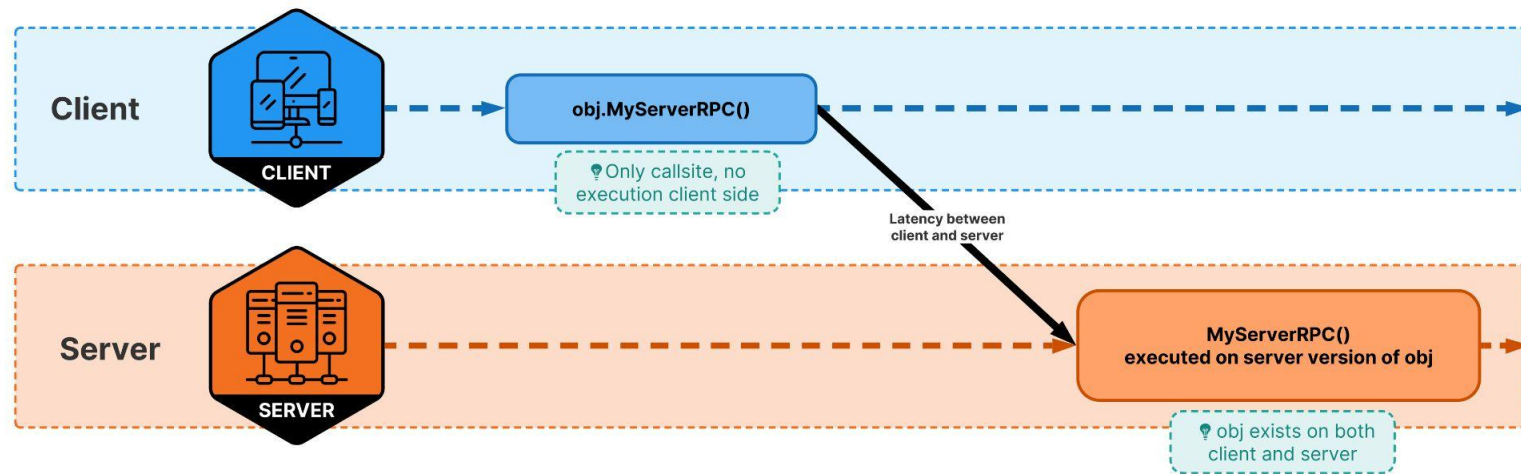
RPC

REMOTE PROCEDURE CALL

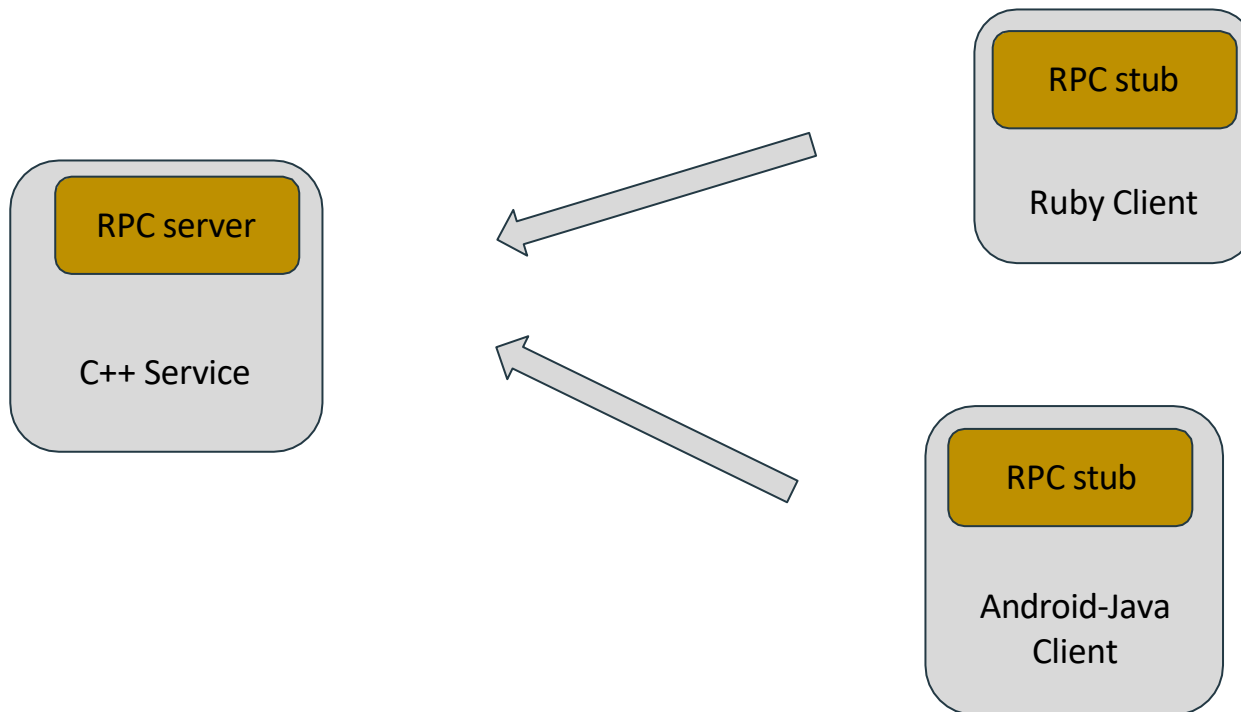
- Poziv udaljene procedure

RPC predstavlja mehanizam za poziv procedura na udaljenim mašinama. Klijent je program u okviru koga se poziva procedura a server onaj koji nudi procedure.

Server RPCs

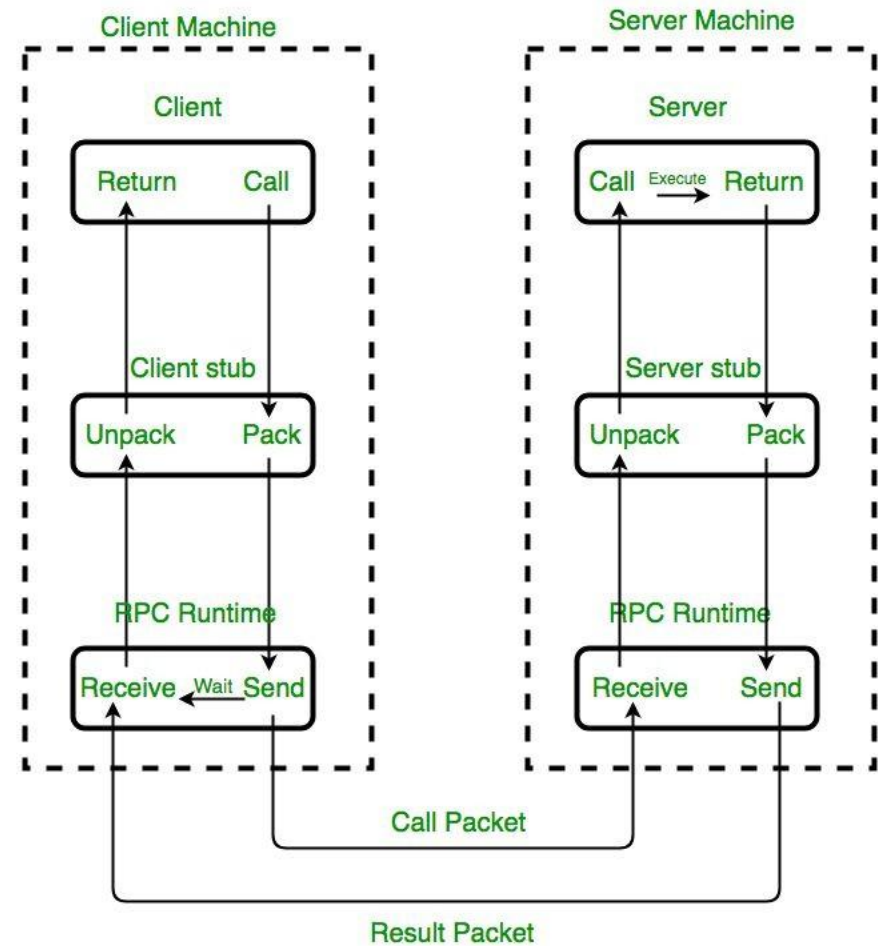


Zašto RPC?



RPC komunikacija

- Da bi poziv udaljene procedure bio moguć izgledao kao poziv lokalne procedure koriste se *stub* funkcije.
- *Stub* funkcija ima isti interfejs kao udaljena funkcija. *Stub*-ovi sa klijent (server) strane su zaduženi za pakovanje (marshalovanje) parametara (rezultata) u mrežnu poruku i slanje preko mreže kao i za raspakovanje (unmarshaling) rezultata (parametara).
- Tom prilikom obavljaju konverzije iz lokalnog u standardni (i obrnuto) način predstavljanja podataka.



Implementation of RPC mechanism

gRPC

gRPC: Google Remote Procedure Call

- Besplatan!
- Adaptacija tradicionalnih RPC okvira
- Koristi HTTP/2 (REST API koristi HTTP/1.1)
- Tight coupling između servera i klijenata
- Streaming & Flow-Controlled
- First Class timeout, lameducking, load-balancing, cancellation,... (grpclb protocol)
- Podrška za 11 programskih jezika
- Kompletne stekove za C/C++, Java i Go, ostale implementacije *wrap*-uju C-
Runtime biblioteke
- Slojevit i nadogradiv dizajn
- Binarni format podataka olakšava komunikaciju
 - Protocol buffers

gRPC

Protocol buffers

Protocol buffers



- Fleksibilni, automatizovani mehanizam za serijalizaciju strukturiranih podataka
- *Language-neutral, Platform-neutral*
- Osnovne prednosti:
 - Backward kompatibilnost
 - Efikasno enkodiranje (3 do 10 puta manja velicina, 20-100 puta ubrzanje)
 - Jaka tipizacija
 - Automatsko generisanje koda za parsiranje



VS



```
<person>
  <name>John Doe</name>
  <email>jdoe@example.com</email>
</person>
```

69 B

```
Person {
  name: "John Doe"
  email: "jdoe@example.com"
}
```

28 B

Način rada

- Definisanje struktuiranog formata podataka u descriptor fajlu
 - (**.proto fajl**)
- Pokretanje protobuf kompajlera za odgovarajući programski jezik nad .proto fajlom za generisanje klasa
- Generisane klase sadrže:
 - Jednostavne funkcije za pristup svakom polju
 - Metode za serijalizaciju i parsiranje cele strukture u/iz raw bajt formata

Primer proto fajla

```
message Person {  
    required string name = 1;  
    required int32 id = 2;  
    optional string email = 3;  
    enum PhoneType {  
        MOBILE = 0;  
        HOME = 1;  
        WORK = 2;  
    }  
    message PhoneNumber {  
        required string number = 1;  
        Optional PhoneType type = 2;  
    }  
    repeated PhoneNumber phone = 4;  
}
```

Message

- Jednostavna sintaksa za definisanje poruka
- Polja u poruci moraju biti označena numeričkim indeksom
- Polja imaju ime, tip i deskriptor (da li je obavezno polje ili ne)
- Poruke mogu importovati ili sadržati druge poruke

Polja

- <field_type, data_type, field_name, encoding_value, [default_value]>
 - required string name = 1;
- Tipovi polja
 - Required
 - Optional
 - Repeated - dinamički nizovi
- Tipovi podataka:
 - Primitivni tipovi
 - Enumerisani tipovi
 - Ugnježdene poruke (Ovim se omogućava pravljenje hijerarhije podataka)
- Encoding
 - Jedinstveni broj (=1, =2,...) koji predstavlja tag koji polje ima u binarno enkodiranoj poruci

gRPC - definisanje servisa

```
service imeServisa {  
    rpc metoda(argumenti) returns (povratni tip);  
}
```

```
service SearchService {  
    rpc Search(SearchRequest) returns (SearchResponse);  
}
```

gRPC

gRPC servisi

- gRPC servisi se definišu u običnim proto fajlovima sa RPC parametrizovanim fajlovima gde su povratni tipovi specificirani kao protobuf poruke
- gRPC koristi **protoc** sa specijalnim gRPC plugin-om za generisanje koda iz proto fajla
- Generiše se gRPC klijent i server kod, kao i regularni protobuf kod za populisanje, serijalizaciju i dobavljanje različitih tipova poruka
- Literatura: <https://developers.google.com/protocol-buffers/docs/overview>
- **Počevši od definicije servisa u .proto fajlu, gRPC obezbeđuje pluginove za kompajliranje protobufera koji generišu klijentski i serverski kod.**
- **Korisnici često zovu ove API-je na klijentskoj strani i implementiraju odgovarajuće API-je na serverskoj strani**

Definicija servisa

- Kao i većina drugih RPC sistema, gRPC je baziran na ideji definicije servisa koja uključuje specifikaciju metoda koje mogu biti pozvani remotely sa određenim parametrima.
- Iako koristi protocol buffers kao IDL, moguće je korišćenje i drugih alternativa
- Synchronous vs. asynchronous
- Moguće je definisati četiri tipa servisnih metoda:
 - **Unary RPC** – klijent šalje jedan zahtev serveru i dobija jedan odgovor
 - **Server streaming RPC** – klijent šalje zahtev serveru i dobija tok odakle može da čita sekvencu poruka
 - **Client streaming RPC** – klijent šalje sekvencu poruka serveru preko toka, server vraća jedan odgovor
 - **Bidirectional streaming RPC** – obe strane šalju sekvence poruka preko read-write toka

RPC life cycle - Unary RPC

- Kada klijent pozove stub metodu, server dobija notifikaciju da je pozvan RPC sa metapodacima klijenta
- Server može da vrati svoje metapodatke odmah (pre bilo kakvog odgovora) ili da čeka klijentski zahtev. Od aplikacije zavisi i implementacija.
- Jednom kada server primi klijentski zahtev, obrađuje ga i popunjava odgovor. Ako je uspešno, odgovor se vraća klijentu zajedno sa statusnim detaljima (status kod i opcionalna poruka) i opcionim metapodacima.
- Ako je status odgovora OK, klijent dobija odgovor čime se kompletira poziv na klijentskoj strani.

RPC life cycle - Server streaming RPC

- Slično kao kod unary RPC
- Server sada vraća niz poruka kao odgovor na klijentov zahtev
- Nakon što pošalje sve poruke, status servera i opcioni metapodaci se šalju klijentu. Ovim se kompletira procesiranje na serverskoj strani
- Klijent kompletira metodu kada primi sve poruke koje je server poslao

RPC life cycle - Client streaming RPC

- Slično kao kod unary RPC
- Klijent sada šalje niz poruka kao zahtev serveru
- Server odgovara jednom porukom zajedno sa statusom i opcionim metapodacima
- Server uglavnom šalje odgovor nakon što primi sve poruke klijenta

RPC life cycle - Bidirectional streaming RPC

- Komunikacija se inicira kada klijent pozove metodu a server primi metapodatke klijenta, ime metode i rok.
- Server može odgovoriti slanjem inicijalnih metapodataka ili čekati da klijent započne slanje.
- Procesiranje podataka i sa klijentske i sa serverske strane je specifično za aplikacije.
- Dva toka podataka su nezavisna, pa klijent i server mogu čitati i pisati poruke u bilo kom redosledu.

Kreiranje gRPC servera

1. Neophodni NuGet paketi:
 - Grpc.AspNetCore
 - Grpc.Tools
 - Google.Protobuf
2. Napraviti folder Protos i u njemu napraviti .proto fajl (*file.proto*)
3. Ažurirati .csproj fajl

```
<ItemGroup>  
    <Protobuf Include="Protos/file.proto" GrpcServices="Server" />  
</ItemGroup>
```
4. Implementirati servis
5. Dodati novokreirani servis u Program.cs

Kreiranje gRPC klijenta

1. Neophodni NuGet paketi:
 - Grpc.Net.Client
 - Grpc.Tools
 - Google.Protobuf
2. Napraviti folder Protos i iskopirati serverski .proto fajl u njega
3. Ažurirati .csproj fajl

```
<ItemGroup>  
  <Protobuf Include="Protos/file.proto" GrpcServices="Client" />  
</ItemGroup>
```
4. Implementirati klijenta

Primeri

[Create a .NET Core gRPC client and server in ASP.NET Core | Microsoft Learn](#)